

บทที่ 2

Web Services

ในบทนี้จะอธิบายถึงรายละเอียดความเป็นมาของเทคโนโลยีเว็บแต่เดิมก่อนที่จะพัฒนามาเป็นเทคโนโลยีเว็บเซอร์วิส ความหมายของเว็บเซอร์วิส และ เทคโนโลยีอื่น ๆ ที่เกี่ยวข้องกับเทคโนโลยีเว็บเซอร์วิส

2.1 ความเป็นมาของการพัฒนาเทคโนโลยีเว็บเซอร์วิส

เทคโนโลยีเว็บอาจแบ่งการประยุกต์และพัฒนาออกได้เป็น 3 ยุคสำคัญ ดังนี้

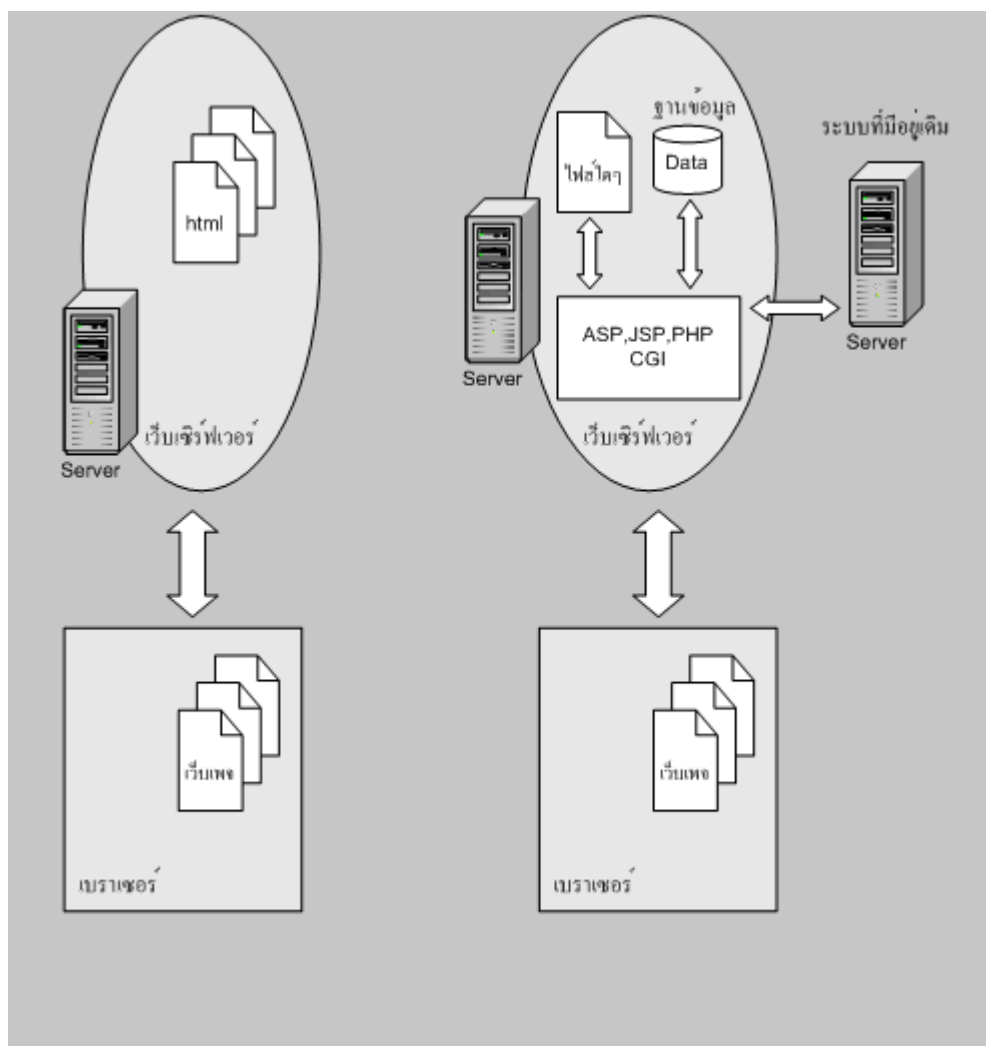
2.1.1 Static Web

ยุคเริ่มต้นของเว็บนั้นเป็นยุคที่มีการใช้เบราว์เซอร์เรียกเว็บเพจที่สร้างด้วยภาษา HTML ทั้งหมดหรือมีสคริปต์ทางฝั่งไคลเอนต์ ตัวอย่างเช่น JavaScript, VBScript เป็นต้น ว่าเป็นไฟล์บนเซิร์ฟเวอร์ ผู้ใช้งานเรียกดูผลการประมวลผลผ่าน HTTP Protocol ในรูปแบบ HTML ซึ่งไฟล์เว็บเพจที่เป็นสคริปต์เหล่านี้ไม่สามารถติดต่อกับองค์ประกอบอื่น ๆ ของทางฝั่งเซิร์ฟเวอร์ได้ เนื่องจากคุณลักษณะของเว็บเพจมีลักษณะคงที่ หรือมีการเปลี่ยนแปลงน้อย จึงเรียกว่า ยุค Static Web

2.1.2 Dynamic Web

เนื่องด้วยรูปแบบการแสดงผลและประมวลผลแบบ Static Web นั้นตายตัว จึงมีการพัฒนาให้เว็บมีความสามารถในการติดต่อกับองค์ประกอบอื่น ๆ ทางฝั่งเซิร์ฟเวอร์ โดยทำการพัฒนาโปรแกรมทางฝั่งเซิร์ฟเวอร์ พัฒนาให้เซิร์ฟเวอร์ทำงานตามการเรียกขอบริการของไคลเอนต์ผ่านทางโปรโตคอล HTTP และให้มีการเชื่อมโยงกับโปรแกรมเฉพาะตามที่ทำการเขียนสคริปต์ขึ้นมา หรือเขียนสคริปต์ขึ้นเพื่อใช้ความสามารถในการประมวลผลของเซิร์ฟเวอร์เพื่อทำงานบางอย่าง เช่น สร้างห้องสนทนา (chat room) กระดานถาม-ตอบ (webboard) เป็นต้น โดยมีการใช้เทคโนโลยี CGI (Common Gateway Interface) ในการทำ dynamic web CGI คือ โปรแกรมที่ทำงานอยู่บนฝั่งเซิร์ฟเวอร์ เมื่อผู้ใช้มีการเรียกใช้ CGI เมื่อใด CGI เมื่อใด CGI ก็จะทำงานตามหน้าที่ที่ถูกเขียนสคริปต์ขึ้นมา ซึ่งอาจมีการส่งผลลัพธ์ตอบกลับไปยังผู้ใช้ หรือไม่มีการส่งผลลัพธ์ไปยังผู้ใช้โดยขึ้นอยู่กับว่ามีการระบุหน้าที่ไว้ในสคริปต์ CGI ไว้เป็นอย่างไร ต่อมาจึงมีการพัฒนาเทคโนโลยีหลายอย่างที่มีหลักการคล้ายกับ CGI เพื่อทำงานทางฝั่งเซิร์ฟเวอร์ เช่น ASP(Active Server Pages), JSP(JavaServer Pages), PHP การพัฒนาโดยใช้เทคโนโลยี CGI เรียกว่า ยุค Dynamic Web

ดังนั้นความแตกต่างระหว่าง Static Web กับ Dynamic Web จะแตกต่างกันทั้งรูปแบบการทำงาน และการประมวลผล



รูปภาพที่ 2.1 เปรียบเทียบลักษณะการทำงานระหว่างเทคโนโลยี Static Web และ Dynamic Web

2.1.3 Web Services

การพัฒนาเว็บพัฒนามาจนมีความสามารถที่เว็บสามารถเรียกใช้แอปพลิเคชันหรือโปรแกรมซึ่งทำงานในลักษณะให้บริการ ผ่านเว็บได้

เว็บเซอร์วิส คือ แอปพลิเคชันหรือโปรแกรมซึ่งทำงานอย่างใดอย่างหนึ่งในลักษณะการให้บริการ โดยจะถูกเรียกใช้งานจากแอปพลิเคชันหรือโปรแกรมอื่นๆ ผ่านเว็บ การให้บริการของเว็บเซอร์วิสจะมีเอกสารที่อธิบายคุณสมบัติของบริการกำกับไว้ และมีการนำเสนอให้สาธารณชนรับทราบ ผู้ใช้บริการจึงสามารถค้นหาเว็บเซอร์วิสได้โดยไม่ต้องรู้ที่อยู่จริงของแอปพลิเคชันหรือโปรแกรมนั้น

ตัวอย่างเช่น การดำเนินการธุรกิจการค้าต่างๆ ซึ่งมีลักษณะงานที่ต้องทำงานร่วมกันระหว่างองค์กร (Interoperability) โดยให้แอปพลิเคชันหรือโปรแกรมขององค์กรหนึ่งส่งคำขอผ่านเครือข่ายอินเทอร์เน็ตด้วย HTTP Protocol ไปยังเว็บบริการของอีกองค์กรหนึ่ง แล้วมีการโต้ตอบเพื่อรับ-ส่งข้อมูลระหว่างกันแบบอัตโนมัติได้ เป็นต้น ยุคที่ซอฟต์แวร์ คือ บริการที่มีอยู่บนเว็บ เรียกว่า ยุค Web Services เว็บเซอร์วิสเป็นแอปพลิเคชันที่ช่วยให้สามารถเรียกใช้ข้อมูลบนเว็บ ได้เหมือนกับการเรียกโปรแกรมที่ใช้ประมวลผลข้อมูลภายในองค์กร ซึ่งช่วยเพิ่มความสามารถในการเรียกใช้ข้อมูลบนอินเทอร์เน็ตได้ยิ่งขึ้น

2.2 แนวคิดของเทคโนโลยีต่าง ๆ ที่นำมาสู่การพัฒนาเป็นเทคโนโลยีเว็บเซอร์วิส

- การพัฒนาโปรแกรมแบบซอฟต์แวร์คอมโพเนนต์ (Software Component) ตามแนวคิดของการออกแบบและพัฒนาโปรแกรมเชิงวัตถุ (Object-Oriented Concept)
- การออกแบบระบบแบบกระจายศูนย์ (Distributed Computing) ซึ่งเป็นเป้าหมายสำคัญของการพัฒนาระบบตามสถาปัตยกรรมแบบ Client-Server
- การทำ EDI (Electronic Data Interchange) ซึ่งสร้างขึ้นโดยกำหนดรูปแบบและมาตรฐานของข้อมูลสำหรับการทำธุรกิจ
- การบูรณาการของซอฟต์แวร์ต่างระบบ (Enterprise application integration : EAI) ที่อยู่บนพื้นฐานของความต้องการใช้ข้อมูลร่วมกัน รวมทั้งการแลกเปลี่ยนข้อมูลระหว่างแอปพลิเคชันให้สามารถทำงานได้อย่างถูกต้องเหมาะสม
- แนวคิดการทำเหมืองข้อมูล (Data Mining) ซึ่งต้องการนำข้อมูลที่ถูกจัดเก็บไว้ในรูปแบบที่แตกต่างกันตามแหล่งต่างๆ มาใช้งานร่วมกัน
- รูปแบบการให้บริการซอฟต์แวร์แบบ ASP (Application Service Provider)

2.3 Web Services

เว็บเซอร์วิส คือ Software Component ที่ผู้ให้บริการนำมาสร้างเป็นแอปพลิเคชันสำหรับให้บริการกับผู้ใช้บริการทางอินเทอร์เน็ต และผู้ใช้บริการสามารถที่จะขอรับบริการจากหลายๆ ที่มาประกอบกันได้ เพราะแต่ละระบบมีความเป็นอิสระจากกัน (Loosely Coupled)

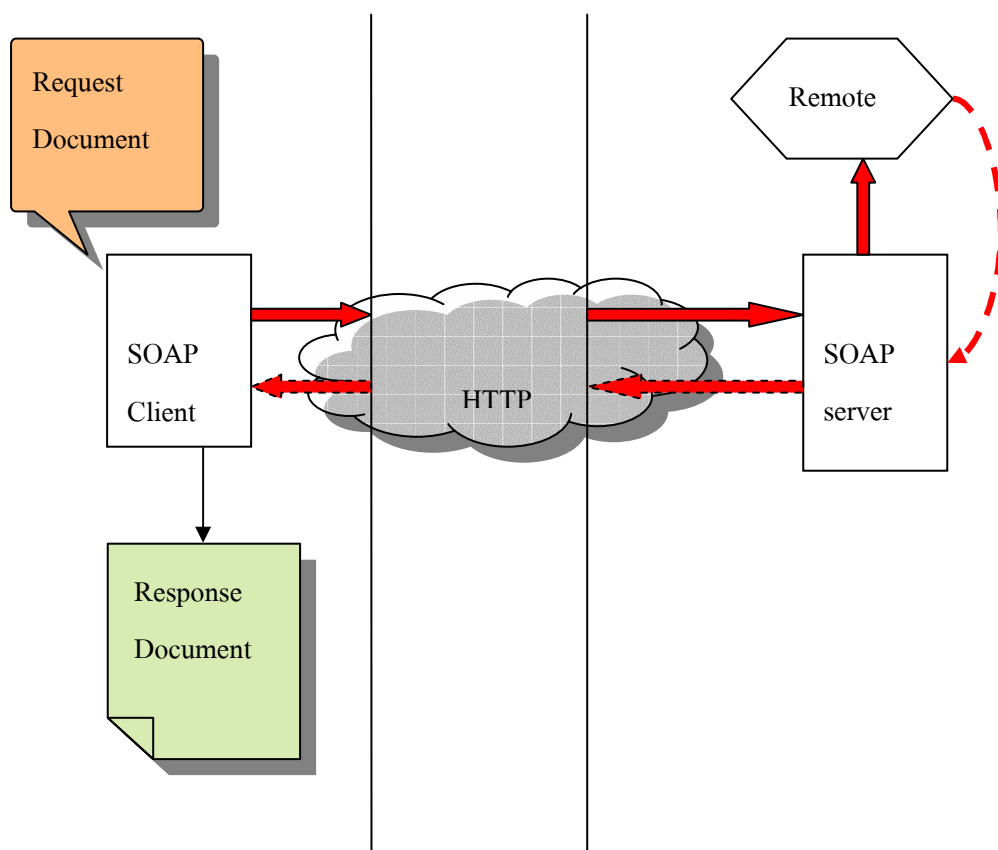
เนื่องจากระบบที่ใช้อยู่ส่วนมากในปัจจุบันเป็นระบบที่ต้องทำงานโดยขึ้นอยู่กับอ็อบเจกต์ (Tightly Coupled) ตัวอย่างเช่น บนแพลตฟอร์ม UNIX อ็อบเจกต์ที่สามารถนำกลับมาใช้ใหม่ได้จะอยู่ในรูปของ Shared Objects ซึ่งจะไม่สามารถนำมาใช้บนแพลตฟอร์มวินโดวส์ได้ ในลักษณะนี้คือระบบที่ทำงานโดยขึ้นอยู่กับอ็อบเจกต์

ระบบที่ทำงานโดยขึ้นอยู่กับอ็อบเจกต์ (Tightly Coupled) นั้นมีปัญหาในการสื่อสารข้ามแพลตฟอร์ม โปรแกรมบนวินโดวส์สามารถเรียกใช้อ็อบเจกต์นั้นจะต้องอยู่บนเครื่องเดียวกับโปรแกรม เช่น เว็บเซิร์ฟเวอร์อาจให้บริการ (บริการเป็นอ็อบเจกต์ COM) โดยเชื่อมต่อไปยังฐานข้อมูลแบบโลคอล (Local) สำหรับเรียกข้อมูลหุ้นขึ้นมาใช้งาน การทำงานจะไม่มีปัญหาใดที่อ็อบเจกต์ COM และโปรแกรมที่เรียกใช้งานนั้นอยู่บนเครื่องเดียวกัน ต่างกับบริการสมัยใหม่ที่ฐานข้อมูลอยู่ที่หนึ่งและโปรแกรมที่เรียกใช้งานอยู่อีกที่หนึ่ง ฉะนั้นไมโครซอฟท์จึงได้พัฒนาสถาปัตยกรรมที่เรียกว่า Distributed Component Object Model (DCOM) สถาปัตยกรรมนี้ช่วยให้เราสามารถเรียกใช้อ็อบเจกต์ แม้อ็อบเจกต์และโปรแกรมจะไม่ได้อยู่บนเครื่องเดียวกัน แต่ขอให้มีการเชื่อมต่อเดียวกันเท่านั้น อ็อบเจกต์ DCOM ได้รับการพัฒนาให้สามารถใช้บนแพลตฟอร์มอื่นได้ด้วย กระนั้นการสื่อสารระหว่างแพลตฟอร์มที่ใช้อ็อบเจกต์ DCOM ยังทำได้ยากอยู่ดี เช่น อ็อบเจกต์ DCOM ไม่สามารถเรียกใช้อ็อบเจกต์ EJB บน Linux ได้ มีบริษัทมากมายทั่วโลกที่มีข้อมูลรอให้บริการกับผู้ใช้ แต่ถ้าบริษัทเหล่านั้นจะต้องทำการเรียกใช้อ็อบเจกต์โดยตรงเอง ก็

เกิดปัญหาการเรียกใช้อ็อบเจกต์ข้ามแพลตฟอร์ม ในกรณีที่ไม่สามารถใช้ Microsoft Visual Basic ไปติดต่อเรียกใช้ EJB ได้ เช่นเดียวกับที่ EJB ก็ไม่สามารถเขียนโปรแกรมติดต่อเรียกใช้ COM ได้

ระบบที่เป็นอิสระต่อกันจึงเกิดขึ้นมา (Loosely Coupled) โดยตั้งอยู่บนสถาปัตยกรรม Simple Object Access Protocol หรือ SOAP ที่พัฒนาขึ้น สถาปัตยกรรม SOAP ช่วยแก้ปัญหาการติดต่อสื่อสารข้ามแพลตฟอร์ม โดย SOAP เป็นไวยากรณ์ที่อนุญาตให้สร้างแอปพลิเคชันเพื่อเรียกใช้อ็อบเจกต์จากระยะไกล SOAP ทำให้สามารถเรียกใช้อ็อบเจกต์และโปรแกรมที่เรียกไม่จำเป็นต้องอยู่บนแพลตฟอร์ม หรือใช้ภาษาโปรแกรมมิ่งเดียวกัน SOAP ใช้ไวยากรณ์มาตรฐานเปิดในการเรียกใช้เมธอดแทนการเรียกเมธอดด้วยโปรโตคอลไบนารีเหมือนในระบบ Tightly Coupled แบบเดิม ซึ่งมาตรฐานนั้น คือ XML

ข้อมูลระหว่างแอปพลิเคชันที่มีการร้องขอและอ็อบเจกต์ที่ได้รับสามารถเป็นข้อมูลแท็กในกระบวนการของ XML (XML Stream) ซึ่ง Stream ดังกล่าวคือข้อความธรรมดาที่สามารถส่งผ่าน HTTP และไฟล์วอลล์ได้



รูปภาพที่ 2.2 SOAP นำเสนอวิธีการใช้อ็อบเจกต์เมธอดผ่าน HTTP

จากรูปภาพที่ 2.2 ผู้ใช้ (SOAP Client) จะส่งเอกสารผ่าน HTTP ไปยังผู้รับ (SOAP Server) โดยที่ผู้รับจะตีความ และทำงานตามที่ต้องการ จากนั้นจึงส่งผลลัพธ์ที่ได้กลับไปยังผู้ใช้ที่รออยู่ โดยขั้นตอนการทำงานจะเริ่มจาก โปรแกรมในฐานะ SOAP Client สร้างเอกสาร XML สำหรับเรียกใช้เมธอดจากระบบภายนอก ขอให้ถึง SOAP Client ในลักษณะต่างจากไคลเอนต์ปกติทั่วไป เช่น SOAP Client อาจ

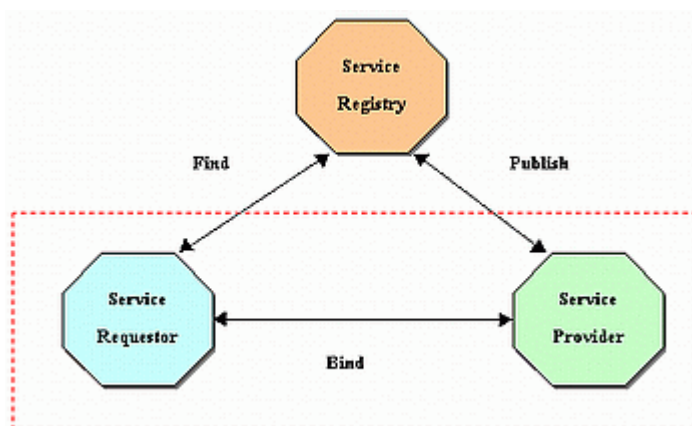
เป็นเว็บเซิร์ฟเวอร์ หรือโปรแกรมบนเดสก์ท็อป สรุป SOAP Client คือสิ่งที่ร้องขอบริการจาก SOAP Server โดย SOAP Client จะครอบคลุมเอกสาร XML ไว้ภายในสิ่งที่เรียกว่า SOAP Envelope และส่งข้อมูลผ่าน HTTP POST Request จากนั้นแพ็คเกจข้อมูลจะถูกส่งผ่านโปรโตคอล HTTP โดยแอปพลิเคชัน (SOAP Server) ปลายทางรอรับข้อมูลอยู่ โดยปกติแอปพลิเคชันนั้น คือ Web Server ซึ่งเมื่อได้รับแพ็คเกจแล้ว แอปพลิเคชัน (SOAP Server) จะตีความเพื่อเรียกใช้เมธอดหรืออ็อบเจกต์ที่เหมาะสมตามข้อมูลที่ส่งมาทำงานตามคำสั่งและส่งผลลัพธ์ที่ได้กลับยัง SOAP Server และ SOAP Server จะแพ็คเกจข้อมูลไว้ใน SOAP Envelope จากนั้นนำส่งกลับไปยังโปรแกรมบน SOAP Client ที่เรียกใช้ผ่าน HTTP เมื่อ SOAP Client ที่รอผลลัพธ์อยู่ได้รับ SOAP Envelope ที่ส่งกลับมาจะนำมาแยกข้อมูลออกมาและส่งไปให้โปรแกรมที่รออยู่

เนื่องจากเอกสาร SOAP ถูกส่งผ่านโดยโปรโตคอล HTTP (พอร์ต 80) จึงสามารถผ่านไฟร์วอลล์ได้เกือบทุกประเภท ไฟร์วอลล์ส่วนใหญ่จะอนุญาตให้ข้อความธรรมดาผ่านโดยไม่ถูกตรวจสอบฉะนั้นการที่จะส่งข้อมูลระหว่างแพลตฟอร์มจึงทำได้โดยไม่ต้องเปลี่ยนแปลงสถาปัตยกรรมของไฟร์วอลล์

ข้อสังเกตที่สำคัญคือจะไม่มีการเปลี่ยนแปลงใด ๆ กับอ็อบเจกต์ที่ถูกเรียกใช้งานในระบบนี้ หน้าที่ของ SOAP Server คือตีความให้อ็อบเจกต์สามารถเข้าใจสิ่งที่ส่งผ่าน HTTP มา SOAP Server เป็นตัวกลางระหว่างระบบทั้งสอง ฉะนั้นเราจึงสามารถเขียนอ็อบเจกต์ด้วยภาษาโปรแกรมมิ่งใดก็ได้ เนื่องจากการสื่อสารทั้งหมดใช้เอกสารที่สร้างจากไวยากรณ์ของ XML

โดย SOAP โปรโตคอลนั้นเป็นโปรโตคอลที่สร้างขึ้นเพื่อแก้ปัญหาคาดต่อข้ามแพลตฟอร์มของสถาปัตยกรรม SOA (Service-Oriented Architecture) โดยการติดต่อสื่อสารระหว่างกันของเว็บเซอร์วิส นั้นยังคงใช้สถาปัตยกรรม SOA แต่เปลี่ยนโปรโตคอลจากเดิมที่อ็อบเจกต์ของแต่ละบริษัทนั้นจะใช้โปรโตคอลแตกต่างกันไป เช่น อ็อบเจกต์ COM (Component Object Model) จะต้องใช้โปรโตคอล DCOM (Distributed Component Object Model) ในการติดต่อสื่อสารระหว่างกัน เป็นต้นมาเป็นโปรโตคอลที่ทุกคนติดต่อกันได้โดยไม่ขึ้นอยู่กับแพลตฟอร์ม

สถาปัตยกรรม SOA (Service-Oriented Architecture) ประกอบด้วยส่วนประกอบหลัก 3 ส่วน คือ ผู้ขอบริการ (Service requester) ผู้ให้บริการ (Service provider) ตัวแทนของผู้ให้บริการ (Service broker)



รูปภาพที่ 2.3 แสดงความสัมพันธ์ของผู้ขอบริการ ผู้ให้บริการ ตัวแทนผู้ให้บริการ ในสถาปัตยกรรม SOA

โดยส่วนประกอบหลักทั้ง 3 ส่วนนี้มีลักษณะการทำงานดังนี้

- ผู้ให้บริการทำการประกาศบริการที่ตนเองให้บริการไปยังตัวแทนของผู้ให้บริการ ซึ่งตัวแทนของผู้ให้บริการจะทำการบันทึกเก็บไว้ในไดเรกทอรีของบริการ (Directory Service)
- ผู้ขอบริการจะทำการค้นหาบริการจากตัวแทนผู้ให้บริการ
- เมื่อพบบริการที่ต้องการ ผู้ให้บริการและผู้ขอบริการจะทำการติดต่อกัน โดยผู้ขอบริการจะทำการเรียกใช้บริการไปยังผู้ให้บริการนั้น

2.4 มาตรฐานต่าง ๆ ที่ใช้ในเว็บเซิร์ฟเวอร์

2.4.1 HTTP (Hyper-Text Transfer Protocol)

เพื่อความเข้าใจเว็บเซิร์ฟเวอร์ที่ใช้ HTTP จึงต้องเข้าใจการทำงานของ HTTP โปรโตคอล HTTP เป็นโปรโตคอลที่ใช้ข้อความธรรมดาจึงมีลักษณะไม่ซับซ้อน สามารถส่งผ่านสื่อกลางอะไรก็ได้ HTTP สื่อสารโดยข้อความที่ส่งกันระหว่างไคลเอนต์และเซิร์ฟเวอร์ ลักษณะอย่างนี้บางครั้งเรียกว่า การร้องขอ (Request) และ การตอบรับ (Response) มาตรฐาน HTTP มีเมธอดให้เลือกใช้มากมายที่นิยมมากที่สุดคือ GET และ POST

เมธอด GET จะถูกโปรแกรมเรียกใช้งานเมื่อผู้ใช้ป้อน URL ในช่องแอดเดรส (Address Bar) ของเบราว์เซอร์ เช่น ผู้ใช้พิมพ์ <http://www.mywebpage.com/index.html> เว็บเบราว์เซอร์จะสร้างแพ็คเกจ HTTP ที่ประกอบด้วยส่วนเฮดเดอร์ของการร้องขอ (Request Header) เพื่อทำการส่งไปยังเซิร์ฟเวอร์ตาม URL ข้างต้น ส่วนเฮดเดอร์ (Header) มีลักษณะต่อไปนี้

```
GET /index.html HTTP/1.1
Host: www.mywebpage.com
Content-Type: text/html
{blank line}
```

แพ็คเกจ HTTP ประกอบด้วยส่วนต่าง ๆ ต่อไปนี้

- บรรทัดเริ่มต้น
- บรรทัดที่ศูนย์หรือบรรทัด Header
- บรรทัดว่าง (เช่น อักขระสำหรับขึ้นบรรทัดใหม่)
- ส่วน Body ที่จะมีหรือไม่มีก็ได้ (เช่น ไฟล์ ข้อมูลคิวรี คิวรีเอาต์พุต)

บรรทัดเริ่มต้นใช้บอกจุดประสงค์ของแพ็คเกจ กรณีนี้เมธอด GET จะบอกเซิร์ฟเวอร์ให้ค้นหาไฟล์ที่ชื่อ /index.html จบบรรทัดด้วยเวอร์ชันของ HTTP (ในตัวอย่างนี้คือ 1.1)

บรรทัดถัดไปคือพารามิเตอร์ของ Header ที่มีลักษณะเป็น Key-value Pairs โดยเริ่มต้นด้วยชื่อพารามิเตอร์ ตามด้วยโคลอน ช่องว่างและค่าของพารามิเตอร์ จบด้วยอักขระสำหรับขึ้นบรรทัดใหม่ (Single Newline Character)

ส่วน HTTP Request นี้สามารถมีพารามิเตอร์ได้หลากหลาย พารามิเตอร์เหล่านี้อาจเป็นชนิดของเบราว์เซอร์ ชนิดระบบปฏิบัติการหรือคุกกี้ แต่ควรระวังว่ายังมีพารามิเตอร์มากที่ยังส่งข้อมูลยังเซิร์ฟเวอร์ได้เข้าส่วน Header จบด้วยอักขระสำหรับขึ้นบรรทัดใหม่จำนวน 2 ตัว ถัดไปจึงเป็นส่วน Body ของแพ็คเกจ HTTP

เมื่อเซิร์ฟเวอร์ได้รับแพ็คเกจ (เว็บเซิร์ฟเวอร์) จะทำการค้นหาไฟล์ (/index.html) เมื่อหาพบจะส่งไฟล์นี้กลับไปยังไคลเอนต์

HTTP Response

HTTP/1.1 200 OK

Date: Fri, 31 Dec 2003 23:59:59 GMT

Content-Type: text/html

Content-Length: 283

<HTML>

<HEAD>

<TITLE>My First Web Page</TITLE>

</HEAD>

<BODY>

<H1>My First Web Page</H1>

<P>This is my first homepage that I tried to write. </P>

<P></P>

</BODY>

</HTML>

โค้ดนี้แสดงแพ็คเกจการตอบรับ (Response Package) ที่มี HTTP Header ตามด้วยอักขระสำหรับขึ้นบรรทัดใหม่จำนวน 2 ตัว และสิ่งที่เรียกว่า Payload โดยทั่วไป Payload คือเอกสาร HTML สังเกตว่าแพ็คเกจการตอบรับมีข้อมูลของตัวเองในบรรทัดแรกของ Header ตามด้วย Key-value Pairs ซึ่งช่วยให้เว็บเบราว์เซอร์ทราบว่าจะต้องทำอะไรกับแพ็คเกจ บรรทัดแรกประกอบด้วยโค้ดแสดงสถานะ ที่พบบ่อยที่สุดคือ 200 ซึ่งหมายถึง ส่งแพ็คเกจเรียบร้อย 404 หมายถึง ไม่พบไฟล์ที่ต้องการ และ 500 หมายถึง เซิร์ฟเวอร์มีปัญหา ส่วนโค้ดอื่น ๆ มีระดับดังนี้

- 1xx หมายถึง ข้อความที่ใช้เป็นข้อมูลเท่านั้น

- 2xx หมายถึง แสดงสถานะความสำเร็จบางอย่าง
- 3xx หมายถึง การส่งต่อไปให้ URL อื่น
- 4xx หมายถึง ความผิดพลาดในส่วนไคลเอนต์
- 5xx หมายถึง ความผิดพลาดในส่วนเซิร์ฟเวอร์

การตอบรับทั้งหมดยกเว้นโค้ดระดับ 100 (แต่รวม Error Response ด้วย) จะต้องมี Date: Header เวลาที่ใช้ใน HTTP ซึ่งจะตรงตามมาตรฐานกรีนิช

ในส่วนเมธอด POST จะทำงานเมื่อผู้ใช้คลิกปุ่ม Submit ในเว็บเพจจะเป็นการส่งข้อมูลไปยังเซิร์ฟเวอร์ HTML มีэлеเมนต์สำหรับรับข้อมูลจากผู้ใช้เพื่อส่งไปยังเซิร์ฟเวอร์ โดยในตัวอย่างนี้เราได้ใช้ элемент INPUT สำหรับรับข้อมูลจากผู้ใช้และใช้ элемент POST สำหรับส่งข้อมูลไปยังเซิร์ฟเวอร์

```
<HTML>

  <HEAD>

    <TITLE>1040 Super E-Z Form </TITLE>

  </HEAD>

  <BODY STYLE="font-family:Verdana;">

    <H1>1040 Super E-Z form</H1>

    <FORM METHOD = "POST" ACTION = "/cgi-bin/processForm.pl">

      <TABLE>

        <TR>

          <TD>Enter your social security number:</TD>

          <TD><INPUT TYPE="TEXT" NAME="socSecNum"></TD>

        </TR>

        <TR>

          <TD>How much did you made last year?</TD>

          <TD><INPUT TYPE="TEXT" NAME="income"></TD>

        </TR>

        <TR>

          <TD>Press "Calculate" for verdict</TD>

          <TD><INPUT TYPE="SUBMIT" VALUE="Calculate"></TD>

        </TR>

      </TABLE>

    </FORM>

  </BODY>

</HTML>
```


สังเกตเอเลเมนต์ FORM เอเลเมนต์นี้จะต้องมีเอเลเมนต์อื่นอยู่ภายใน เช่น INPUT, TEXTAREA, SUBMIT ในตัวอย่างนี้ FORM มีแอตทริบิวต์อยู่ 2 ตัวคือ METHOD และ ACTION แอตทริบิวต์ METHOD ใช้ส่งข้อมูลไปยังเซิร์ฟเวอร์ ส่วนพารามิเตอร์ที่ไม่มีไฟล์นามสกุล .pl หมายถึงสคริปต์ของ Perl ใน /cgi-bin ที่จะถูกรัน เมื่อโหลดไฟล์ในโค้ดจะปรากฏช่องให้ป้อนค่า และปุ่มให้คลิก เมื่อผู้ใช้ป้อนค่าและคลิกปุ่ม Calculate เว็บเบราว์เซอร์จะสร้างแพ็กเก็ต HTTP Information ดังที่แสดงนี้

```
POST /cgi-bin/processForm.pl HTTP/1.1
Host: www.irs.gov
Content-Type: application/x-www-form-urlencoded
Content-Length: 35

socSecNum=123-45-6789&income=80000
HTTP Package ที่รับมาจากผู้ใช้ผ่านฟอร์ม HTML
```

ส่วน Content-type Header จะบอกให้เซิร์ฟเวอร์ทราบว่าแพ็กเก็ตมีการเข้ารหัสข้อมูล ฉะนั้นเซิร์ฟเวอร์จะต้องถอดรหัสข้อมูลก่อนทำการส่งไปให้แอปพลิเคชัน ส่วน Body ของแพ็กเก็ตจะประกอบด้วยฟิลด์อินพุต และค่าที่ผู้ใช้ป้อนเข้ามา มีเครื่องหมายแอมเพอร์แซนด์ (&) คั่นระหว่างค่าเหล่านี้ เมื่อเซิร์ฟเวอร์ได้รับแพ็กเก็ต มันจะส่งค่าในส่วน Body ให้สคริปต์ จากนั้นสคริปต์จะทำการโปรเซสและส่งผลลัพธ์กลับผ่าน HTTP Server

2.4.2 พอร์ต

ในการติดต่อกับเซิร์ฟเวอร์จะใช้พอร์ตเพื่อทำงานในลักษณะต่าง ๆ เช่น FTP Server ใช้พอร์ต 21 เป็นค่าดีฟอลต์ HTTP Server ใช้พอร์ต 80 พอร์ตที่กำหนดเหล่านี้สามารถเปลี่ยนแปลงได้แต่ไม่ควรทำเนื่องจากแอปพลิเคชันส่วนใหญ่จะเดาว่าสามารถใช้คำร้องขอของ HTTP GET หรือ POST ผ่านพอร์ต 80

HTTP อีกรูปแบบหนึ่งคือ HTTPS เป็นโพรโตคอลที่มีการเข้ารหัสเพื่อความปลอดภัยระหว่างเซิร์ฟเวอร์และเบราว์เซอร์ HTTPS นี้จะใช้พอร์ต 443 เว็บเซิร์ฟเวอร์จะเข้ารหัสข้อมูลเมื่อเบราว์เซอร์ร้องขอทุกครั้งที่มีการแลกเปลี่ยนข้อมูลระหว่างไคลเอนต์และเซิร์ฟเวอร์ ผู้ส่งจะเข้ารหัสข้อมูลและผู้รับจะถอดรหัสข้อมูลนั้นโดยใช้กุญแจที่ทั้ง 2 ฝ่ายมีอยู่

บริการอื่น ๆ ก็มีพอร์ตที่เป็นค่าดีฟอลต์เป็นของตัวเอง เช่น RealServer จาก RealNetworks ใช้พอร์ต 554 และ 700 (อาจเป็นพอร์ต 80 หากพอร์ตที่กล่าวมาถูกใช้ไปแล้ว) Telnet ใช้พอร์ต 23 ผู้ดูแลระบบสามารถใช้ไฟร์วอลล์หรือฟร็อกซีสำหรับจำกัดการเข้าใช้งาน โดยการปิดไม่ให้มีการใช้พอร์ตที่ต้องการ

2.4.3 ไฟล์วอลล์

เหตุผลที่กล่าวถึงสถาปัตยกรรมของ HTTP เนื่องจาก XML สามารถถูกส่งผ่านพอร์ตเหล่านี้ได้ บริษัทสมัยใหม่อนุญาตให้พนักงานใช้อินเทอร์เน็ตได้ เนื่องจากเว็บไซต์ส่วนใหญ่ใช้พอร์ต 80 สำหรับรับส่งข้อมูล ไฟล์วอลล์ส่วนใหญ่จึงอนุญาตให้ข้อมูลผ่านพอร์ตนี้ได้อย่างเสรี โดยไม่มีการตรวจสอบชนิดของข้อมูล เนื่องจากมีชนิดข้อมูลเกิดใหม่เป็นจำนวนมาก การใช้พอร์ต 80 เพื่อสื่อสารกัน จึงเป็นการประหยัดและเป็นการช่วยให้ผู้ดูแลระบบมีงานน้อยลง

เมธอด POST ช่วยให้การส่งข้อมูลระหว่างไคลเอนต์และเซิร์ฟเวอร์ผ่านสถาปัตยกรรมเว็บง่ายขึ้น เนื่องจาก XML อิงกับมาตรฐาน HTML ฉะนั้นจึงสามารถใช้ XML เพื่อส่งผ่านระหว่างเซิร์ฟเวอร์ได้สะดวก

2.4.4 SOAP (Simple Object Access Protocol)

SOAP เป็นโพรโทคอลที่ทำให้ทุกชนิดบนแพลตฟอร์มที่ต่างกันสามารถติดต่อสื่อสารกันได้ง่ายขึ้น โดยใช้ไวยากรณ์ภาษา XML

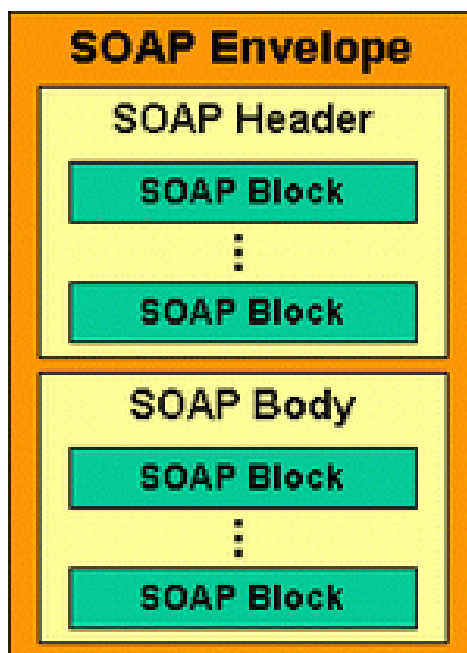
ปัจจุบันแอปพลิเคชันแบบกระจายกันทำงาน (distributed application) ในปัจจุบันใช้ Remote Procedure Call (RPC) สื่อสารกันระหว่างออบเจกต์ เช่น DCOM (Distributed Component Object Model), CORBA (Common Object Request Broker Architecture) เป็นต้น ซึ่งโพรโทคอลเหล่านี้ไม่ได้ใช้งานบน HTTP โพรโทคอล ดังนั้น RPC จึงนำมาปรับใช้กับอินเทอร์เน็ตได้ยาก

นอกจากนี้ RPC ยังมีปัญหาเรื่องความปลอดภัย โดยไฟร์วอลล์ และฟร็อกซีเซิร์ฟเวอร์ จะไม่ยอมให้ส่งข้อมูลชนิดนี้ได้ตามปกติ SOAP ถูกสร้างขึ้นมาเพื่อแก้ปัญหานี้ เพราะ SOAP อ้างอิงอยู่บนมาตรฐาน HTTP โพรโทคอล ซึ่งเป็นที่ยอมรับโดยอินเทอร์เน็ตบราวเซอร์ และเซิร์ฟเวอร์ทุกชนิด

SOAP Client คือ โปรแกรมที่สร้างเอกสารบนพื้นฐานของภาษา XML ร้องขอใช้บริการหรือคอมโพเนนต์ที่เรียกว่า SOAP Message โดยมีการรับ-ส่งข้อมูลบน HTTP ทำให้สามารถแลกเปลี่ยนข้อมูลระหว่างแพลตฟอร์มที่ต่างกัน และส่งข้อมูลผ่านไฟล์วอลล์ได้ โดย SOAP Message จะถูกส่งไปยังเซิร์ฟเวอร์ในรูปเอกสาร XML เมื่อส่งไปแล้วทางเซิร์ฟเวอร์ก็จะมีตัวที่เรียกว่า SOAP Listener เอาไว้คอยดักฟังการร้องขอจากทางไคลเอนต์ หรือผู้ขอใช้บริการ เมื่อทำการประมวลผลเสร็จสิ้น ฟังก์ชันเซิร์ฟเวอร์จะทำการส่งผลลัพธ์ออกมาเป็น SOAP Message เช่นกัน โดยที่เมื่อส่งมาถึงผู้ขอใช้บริการแล้ว ผู้ขอใช้บริการก็สามารถแปลง SOAP Message ที่อยู่ในรูปเอกสาร XML ไปเป็นภาษาที่ผู้ขอใช้บริการต้องการ หรือนำไปแสดงผลได้

2.4.4.1 โครงสร้างของ SOAP Message

โครงสร้างของ SOAP Message มี 3 ส่วนหลักดังนี้



รูปภาพที่ 2.4 แสดงโครงสร้างของเอกสาร SOAP

- SOAP Envelope เนื้อหาสาระ (Content) ของเอกสาร XML ที่ใช้แทน Message
- SOAP Header เป็นส่วนเพิ่มเติมของเอกสาร SOAP ซึ่งจะมีหรือไม่มีก็ได้
- SOAP Body เป็นส่วนที่ใช้เก็บข้อมูลสำหรับส่งไปให้ผู้รับปลายทาง

เอกสาร SOAP เปรียบเหมือนซองจดหมายอิเล็กทรอนิกส์ที่ใช้เก็บสิ่งที่เรียกว่า Payload ส่วนนี้ประกอบด้วยแท็กที่อธิบายเมธอดและค่าที่เมธอดจำเป็นต้องใช้ รูปภาพที่ 2.4 แสดงแท็กแรกของ SOAP โดยที่ SOAP Envelope ประกอบด้วยสับเอลเมนต์ 2 ตัวคือ SOAP:Header และ SOAP:Body เอลเมนต์ SOAP:Header ประกอบด้วยข้อมูลที่ใช้ในการส่ง ข้อมูลนี้ผู้ใช้เป็นคนกำหนด และส่วนของคอนเทนต์ที่ต้องการให้ทำงาน ในตัวอย่างด้านล่าง เอลเมนต์ SOAP:Header ประกอบด้วยเอลเมนต์ Transaction ที่มีค่าเท่ากับ 5 แอปพลิเคชันจะนำค่านี้ไปใช้งาน สังเกตว่าเอลเมนต์ Transaction มี Namespace ที่ผู้ใช้กำหนดเองชื่อ trans

```
<SOAP:Header>
  <trans:Transactionxmlns:trans:"http://schemas.architag.com/transaction.xsd"
    SOAP:mustUnderstand="1">5</trans:Transaction>
</SOAP:Header>
```

เอลเมนต์ SOAP สามารถมีแอตทริบิวต์โกบอล SOAP:mustUnderstand แอตทริบิวต์ส่วนนี้จะเป็นตัวระบุว่าต้องมี Header Entry หรือไม่ โดยมีค่าแอตทริบิวต์นี้เท่ากับ 1 หรือ 0 เท่านั้น ที่เป็นเช่นนี้เพราะจะทำให้ใช้งานกับข้อกำหนดเดิมได้ด้วย เช่น สามารถส่งเอกสาร SOAP ใหม่ไปยังเซิร์ฟเวอร์ที่ถูกเขียนอยู่ในข้อกำหนดเดิม เอลเมนต์ SOAP:Header นี้จะมีหรือไม่มีก็ได้

элемент SOAP:Header เป็นสับ элементตัวที่ 2 ของ Envelope สำหรับ SOAP Request ส่วน Body จะประกอบด้วยแท็กที่นิยามโดยเมธอดที่ร้องขอ แท็กเหล่านี้มีข้อมูลที่เมธอดจำเป็นต้องใช้งาน สำหรับ SOAP Response элемент SOAP:Body ประกอบด้วยค่าที่สร้างจากผลลัพธ์ของข้อความ (โดยถ้าเกิดความผิดพลาดขึ้นนั้นจะรายงานไปที่ элемент SOAP:Fault элементนี้เป็นส่วนที่จะมีหรือไม่มีก็ได้ สำหรับรายงานปัญหาที่เกิดจากการโปรเซสของเซิร์ฟเวอร์)

```
<soap:Envelope>
  <soap:Body>
    <GetPrice>
      <Item>Rose</Item>
      <Quantity>100</Quantity>
    </GetPrice>
  </soap:Body>
</soap:Envelope>
```

ตัวอย่าง SOAP Message อย่างง่ายของการสอบถามราคาดอกกุหลาบ จำนวน 100 ดอก

2.4.4.2 ข้อดีของ SOAP

ข้อดีของ SOAP คือ SOAP ไม่ขึ้นกับคอมพิวเตอร์เทคโนโลยีแพลตฟอร์ม และภาษาการเขียนโปรแกรมใด ๆ ทำให้สามารถเขียนได้ง่ายและขยายเพิ่มเติมได้

2.4.5 XML (eXtensible Markup Language)

ลักษณะที่สำคัญของภาษา XML

- XML สามารถรองรับการแลกเปลี่ยนข้อมูลระหว่าง application โดยไม่ขึ้นอยู่กับ platform
- XML เป็นภาษาที่ได้รับการออกแบบมาเพื่อให้สามารถนิยามความหมายของข้อมูลได้ จึงมีการจัดโครงสร้างข้อมูล แบ่งข้อมูลออกเป็นหมวดหมู่และส่วนประกอบย่อย
- XML ไม่มี tag ที่ถูกนิยามไว้ก่อน และอนุญาตให้ผู้ใช้สามารถสร้าง tag ขึ้นมาเพื่อใช้อธิบายข้อมูลเองได้ โดย tag ที่สร้างขึ้นผู้สร้างจะเป็นผู้กำหนดนิยาม และความหมายของ tag ตามข้อตกลงของ W3C ซึ่งทำให้การอ่านเอกสารเป็นไปได้ง่ายและเป็นมาตรฐาน

- ส่วนข้อมูล และส่วนการแสดงผลของเอกสาร XML ถูกแยกออกจากกันอย่างชัดเจน โดยที่ในเอกสาร XML นั้นจะมีแต่ตัวเนื้อข้อมูล จึงทำให้การเปลี่ยนแปลงข้อมูลไม่ส่งผลกระทบต่อ การแสดงผล และการแก้ไขส่วนแสดงผลก็จะไม่ส่งผลกระทบต่อตัวเนื้อข้อมูล และการนำข้อมูลไปแสดงผลนั้นก็มิให้เลือกมากมาย เช่น CSS XSL HTML เป็นต้น

- การอ่านและแปลความหมายของเอกสาร XML สามารถทำได้โดยใช้โปรแกรมที่เรียกว่า XML parser

- XML มีความกะทัดรัด เข้าใจง่าย และใช้ประโยชน์ได้กว้างขวาง

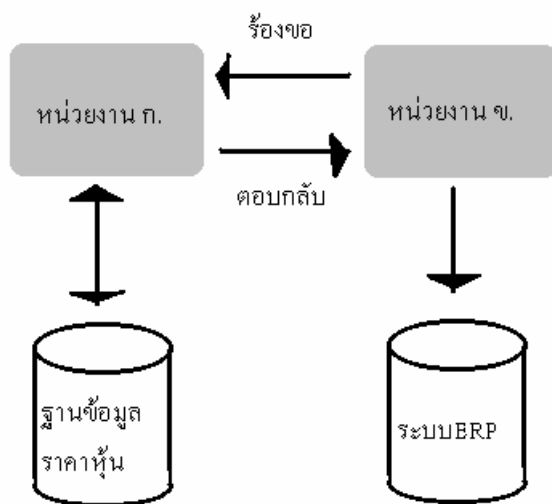
- XML สามารถใช้ได้หลายภาษาผสมกัน เนื่องจาก XML สนับสนุน UNICODE

- XML ได้รับการสนับสนุนในโปรแกรมระบบฐานข้อมูลหลายๆ บริษัทให้สามารถดึงข้อมูลจากฐานข้อมูลมาอยู่ในรูปของ XML ได้

ประโยชน์ของ XML

- ใช้สร้างข้อมูลเพื่ออธิบายความหมายของตัวเองได้

- ใช้สำหรับการแลกเปลี่ยนข้อมูล



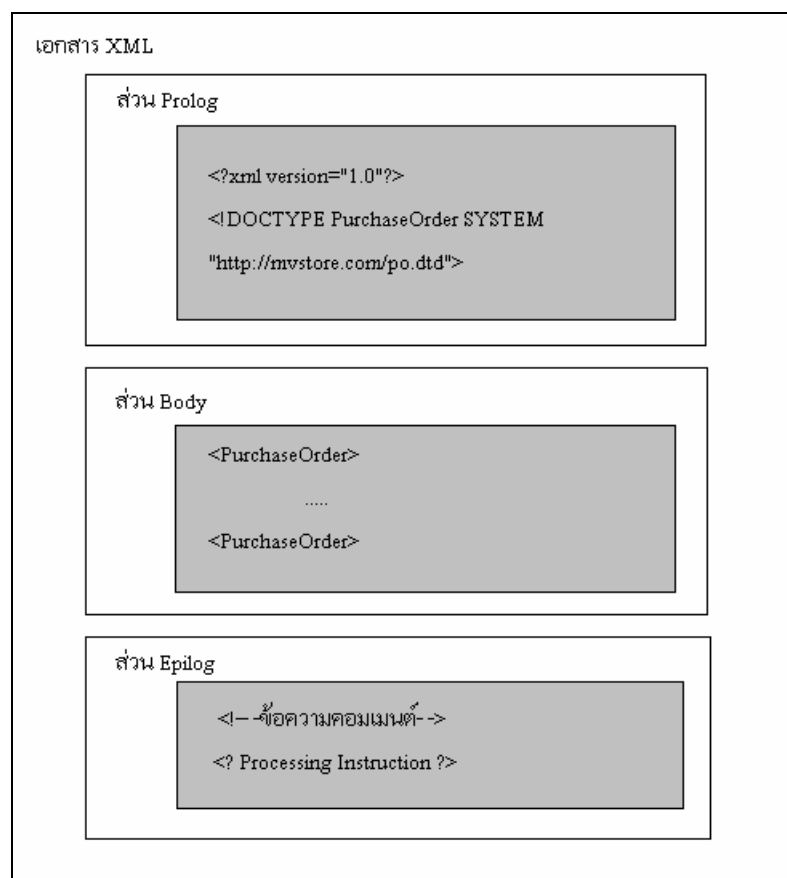
รูปภาพที่ 2.5 ตัวอย่างการแลกเปลี่ยนข้อมูลในรูป XML

จากรูป หน่วยงาน ก. ขอข้อมูลสภาพอากาศจากหน่วยงาน ข. แล้วหน่วยงาน ข. ก็จะได้ข้อมูลราคาหุ้นจากฐานข้อมูลแล้ว markup ด้วยแท็กที่กำหนดขึ้นมา เช่น <Price>950</Price> เป็นต้น เมื่อหน่วยงาน ก. ได้รับข้อมูลแล้วก็สร้างโปรแกรมดึงข้อมูลจริงๆ ในไฟล์ XML ไปใช้งานต่อไป ดังนั้นหน่วยงาน ก. และหน่วยงาน ข. จะต้องตกลงชื่อแท็กกันไว้ก่อน

- เป็นรูปแบบข้อความในการสื่อสาร (messaging format) ระหว่างแอปพลิเคชัน

- เป็นรากฐานของภาษาใหม่ๆ ในการพัฒนาเว็บซึ่งได้แก่ XHTML, MathML(กลุ่มของแท็ก เพื่อใช้ นิยามเครื่องหมายในทางคณิตศาสตร์ขั้นสูง), VML(ภาษาที่ใช้วาดรูปกราฟิกส์เพื่อแสดงผ่านเว็บเบราว์เซอร์) และ WML(ภาษาที่ใช้ในการสร้าง WAP Site) เป็นต้น

โครงสร้าง XML ประกอบด้วย 3 ส่วน คือ



รูปภาพที่ 2.6 ส่วนประกอบของเอกสาร XML

- ส่วน **Prolog** เป็นส่วนที่ใช้ในการ Declaration
 - ส่วน **Body** เป็นส่วนของข้อมูลจริง
 - ส่วน **Epilog** เป็นส่วนที่ข้อความ comment และ PI (Processing Instruction) ซึ่งจริงๆ แล้วจะแทรกอยู่ในส่วน Body ของเอกสาร
- แต่ในภาพนี้แยกไว้ด้านล่างเพื่อให้เห็นว่าในเอกสาร XML มีองค์ประกอบ 3 ส่วน

```
<?xml version="1.0" encoding="windows-874"?>
<?xml-stylesheet type="text/xsl" href="slstyle.xml"?>
<!DOCTYPE StudentList SYSTEM "http://www.myclassroom.com/sl.dtd">
<StudentList>
    <student studentid="1">
        <prefix>นางสาว</prefix>
        <fname>วาสนี</fname>
        <lname>จิ้งชัชวีระยานนท์</lname>
    </student>
```

```

    <student studentid="2">
        <prefix>นาย</prefix>
        <fname>เมธี</fname>
        <lname>สุริยะ ไกร</lname>
    </student>
</StudentList>

```

ตัวอย่างไฟล์ XML

2.4.6 WSDL (Web Services Description Language)

WSDL (Web Services Description Language) เป็นภาษาที่ใช้อธิบายคุณลักษณะการให้บริการของเว็บเซอร์วิส และวิธีการติดต่อขอรับบริการจากเว็บเซอร์วิส เช่น ชื่อเว็บเซอร์วิส ชื่อเมธอดของ COM Component ที่เปิดให้บริการ พารามิเตอร์ที่ต้องส่งไปยังเมธอด ชนิดข้อมูลของพารามิเตอร์ เป็นต้น โดยรายละเอียดเหล่านี้จะเป็นไปตามหลักไวยากรณ์ของภาษา XML (eXtensible Markup Language)

โครงสร้างของ WSDL แบ่งออกเป็น 2 ส่วนใหญ่ๆ คือ

- Abstract Definitions Group ทำหน้าที่กำหนด message SOAP ที่ไม่ขึ้นกับแพลตฟอร์มหรือ รูปแบบภาษา
- Concrete Descriptions Group ทำหน้าที่กำหนดข้อมูลที่ขึ้นกับไซต์ เครื่องและภาษา

```

<?xml version = "1.0"?>
<definitions name = "StockQuoteService" targetNamespace =
    "http://www.xmethods.net/sd/StockQuoteService.wsdl"
    xmlns:tns = "http://www.xmethods.net/sd/StockQuoteService.wsdl"
    xmlns:xsd = "http://www.w3.org/1999/XMLSchema"
    xmlns:soap = "http://schemas.xmlsoap.org/wsdl/soap/"
    xmlns = "http://schemas.xmlsoap.org/wsdl/">
    <message name = "getQuoteRequest">                //จุดสังเกตที่ 3
        <part name = "xsd symbol" type = ":string"/>
    </message>
    <message name = "getQuateResponse">                //จุดสังเกตที่ 4
        <part name = "Result" type = "xsd:float"/>
    </message>
    <portType name = "StockQuotePortType">
        <operation name = "getQuote">
            <input message = "tns:getQuoteRequest" name = "getQuote"/>

```

```

    <output message = "tns:getQuoteResponse" name = "getQuoteResponse"/>
  </operation>
</portType>
<binding name = "StockQuoteBinding" type = "tns:StockQuotePortType">
  <soap:binding style = "rpc" transport = "http://schemas.xmlsoap.org/soap/http"/>
  <operation name = "getQuote"> //จุดสังเกตที่ 2
    <soap:operation soapAction=""/>
    <input>
      <soap:body use = "encoded" namespace = "urn:xmethods-delayed-quotes"
        encodingStyle = "http://schemas.xmlsoap.org/soap/encoding"/>
    </input>
    <output>
      <soap:body use = "encoded" namespace = "urn:xmethods-delayed-quotes"
        encodingStyle = "http://schemas.xmlsoap.org/soap/encoding"/>
    </output>
  </operation>
</binding>
<service name = "StockQuoteService"> //จุดสังเกตที่ 1
  <documentation>Obtains 20-minute delayed quotes from Yahoo</documentation>
  <port name = "StockQuotePort" binding = "tns:StockQuoteBinding">
    <soap:address location = "http://services.xmethods.net:80/ soap"/> //จุดสังเกตที่ 5
  </port>
</service>
</definitions>

```

ตัวอย่างไฟล์ WSDL

WSDL แต่ละไฟล์จะมี 5 จุดที่ควรรู้ โดยเรียงลำดับตามความสำคัญจากมากไปน้อย ดังนี้

- จุดสังเกตที่ 1 บอกชื่อ Web Services
- จุดสังเกตที่ 2 บอกชื่อ method ของ Web Services
- จุดสังเกตที่ 3 และ 4 บอกข้อมูล parameter ของ method และค่าที่ส่งกลับมา ของ Web Services
- จุดสังเกตที่ 5 บอกตำแหน่งของไฟล์ ซึ่งทำหน้าที่เป็น SOAP Listener

ตามทฤษฎีแล้วไฟล์เอกสาร WSDL แต่ละไฟล์จะสามารถอธิบายคุณลักษณะของบริการเว็บเซอร์วิสมากกว่า 1 บริการ

2.4.7 UDDI (Universal Description Discovery and Integration)

UDDI ทำหน้าที่ให้ method สำหรับการสร้าง และการหารายละเอียดของเซอร์วิสใน UDDI Registry ซึ่ง UDDI จะรองรับเซอร์วิสได้หลายประเภท และ UDDI จะไม่ขึ้นกับ WSDL

2.5 การนำเทคโนโลยีเว็บเซอร์วิสมาประยุกต์ใช้ร่วมกับ PHP

PHP เป็นภาษาสคริปต์ที่นิยมใช้สำหรับการพัฒนาเว็บเนื่องจากความสามารถของ PHP ที่สามารถฝังตัวร่วมกับ HTML ภาษา PHP เป็นภาษาคอมพิวเตอร์ที่มีความสามารถเชิงวัตถุและมีไวยากรณ์ที่ใกล้เคียงกับ C, Perl และ Java

ลักษณะสำคัญของ PHP หลาย ๆ อย่างนั้นเป็นประโยชน์สำหรับการพัฒนาเว็บเซอร์วิส เช่น ความสามารถในการเขียนโปรแกรมเชิงวัตถุ ซึ่งในข้อนี้อาจเปรียบเทียบกับไม่ได้กับภาษาที่เริ่มต้นพัฒนามาเพื่อการเขียนโปรแกรมเชิงวัตถุแต่เริ่มแรก แต่ภาษา PHP เองนั้นก็จัดหาเครื่องมือเพิ่มเติมสำหรับการเขียนโปรแกรมเชิงวัตถุ และยังอนุญาต SOAP toolkit แยกออกเป็นกลุ่มของคลาสที่สนับสนุนการติดต่อเว็บเซอร์วิส ข้อดีของภาษา PHP อีกสิ่งหนึ่งคือ การรองรับภาษา XML โดยมีฟังก์ชัน XML ไว้สำหรับการต่อเติม เช่น ส่วนเพิ่มเติม domxml ที่ใช้ไลบรารี libxml เพื่อรองรับ DOM, Xpath และ Xlink

ส่วนเพิ่มเติมที่มีประโยชน์สำหรับการพัฒนาเว็บเซอร์วิส คือ ส่วนเพิ่มเติม CURL (Client URL Library) ที่อนุญาตให้สามารถติดต่อสื่อสารผ่านโพรโทคอลหลากหลายชนิด เช่น HTTP, HTTPS, FTP, telnet และ LDAP ส่วนสนับสนุน HTTPS นั้นเป็นประโยชน์สำหรับการใช้เว็บเซอร์วิสใน PHP เนื่องจากอนุญาตให้เว็บเซอร์วิสไคลเอนต์ติดต่ออย่างปลอดภัยกับเซิร์ฟเวอร์

ผู้เริ่มทดลองใช้ PHP เว็บเซอร์วิส อาจเริ่มจากการทดลองเขียนโค้ดไม่กี่บรรทัดโดยไม่ต้องตั้งค่าภาวะแวดล้อมพิเศษและ ไม่ต้องมีความรู้พิเศษเกี่ยวกับการทำงานของเว็บเซอร์วิสเลยก็สามารถทำได้

PHP ได้รับการพัฒนาให้เหมาะสมกับเว็บแอปพลิเคชัน การพัฒนาเว็บเซอร์วิสโดยใช้ภาษา PHP นั้นสามารถนำคอมโพเนนต์ที่ได้สร้างขึ้นแล้วมาทำการปรับปรุงใช้กับ SOAP ได้ การเปลี่ยนแปลงจากข้อมูลใน HTML ไปเป็น SOAP message ทำได้สะดวกรวดเร็วและเป็นเรื่องง่ายโดยใช้เครื่องมือที่มีอยู่ในปัจจุบัน

เหตุผลที่สนับสนุนการใช้ PHP สำหรับเว็บเซอร์วิสนั้นคือ PHP เป็นทางเลือกเดียวที่มีอยู่สำหรับเว็บไซต์ที่ใช้บริการ PHP โฮสต์ดิงเซอร์วิส และต้องการใช้เว็บเซอร์วิสเพื่อใช้ข้อมูลร่วมกันกับหุ่นส่วนเพื่อเข้าใช้บริการร่วมกันไม่สามารถทำได้ เนื่องจากเว็บไซต์ที่ใช้บริการ PHP โฮสต์ดิงเซอร์วิสไม่มีความสามารถในการควบคุมการติดตั้ง PHP ทำให้ไม่สามารถติดตั้ง Java ไม่มีโปรแกรมที่แปลโปรแกรมภาษา PHP และไม่มีสิทธิในการเริ่มต้น Apache ใหม่ โดยถ้าเว็บไซต์สามารถติดตั้งซอฟต์แวร์ลงบนเซิร์ฟเวอร์ได้นั้นผู้ใช้ก็สามารถใช้และได้รับประโยชน์จากเว็บเซอร์วิสเพียงใช้คำสั่งสั้น ๆ เพียงเซิร์ฟเวอร์นั้นมี PHP

PHP ปัจจุบันนั้นไม่มีมาตรฐาน SOAP รองรับ มีวิธีการที่แตกต่างกันหลายวิธีในการอิมพลีเมนต์ SOAP และ วิธีที่ดีที่สุดคือการเขียนด้วย PHP ทั้งหมด

การใช้นำ PHP มาประยุกต์ใช้กับเทคโนโลยีเว็บเซอร์วิสนั้นเริ่มจากไคลเอนต์ จะทำการร้องขอหรือเรียกใช้งานไฟล์ PHP ที่เก็บในเครื่องเซิร์ฟเวอร์โดยจะนำ request มาใส่ใน SOAP Envelope ก่อนจะทำกา

ส่งผ่าน SOAP โปรโตคอล จากนั้นเซิร์ฟเวอร์จะรับ SOAP Envelope มา จากนั้นจะทำการถอดรหัส นำ SOAP message ออกมาจาก SOAP Envelope แล้วนำ request ที่ได้มาทำการค้นหาไฟล์ PHP ที่ต้องการแล้วทำการประมวลผลไฟล์ PHP ตามที่ไคลเอนต์ร้องขอมา

จากนั้น ถ้าไฟล์ PHP ต้องใช้ข้อมูลในฐานข้อมูล เซิร์ฟเวอร์จะทำการติดต่อกับฐานข้อมูล และนำข้อมูลมาใช้ร่วมกับการประมวลผลด้วย จากนั้นเซิร์ฟเวอร์จะทำการ encrypt ข้อมูลกลับให้อยู่ในรูปแบบ SOAP Envelope แล้วจึงค่อยทำการส่งกลับไปยังไคลเอนต์

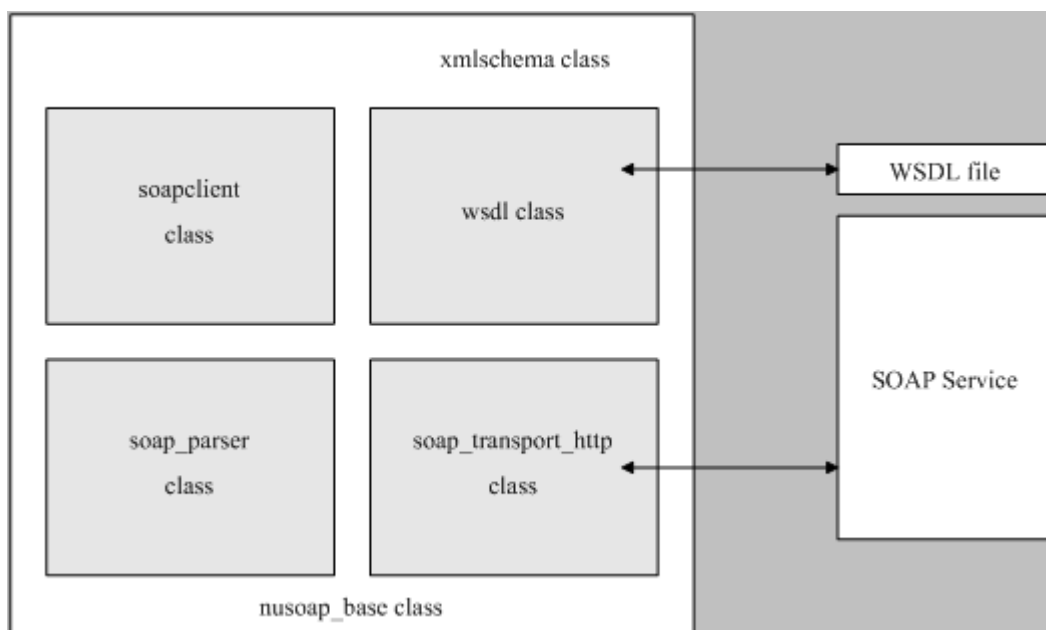
เมื่อไคลเอนต์ได้รับ SOAP Envelope แล้วจะต้องทำการ decrypt เพื่อนำ SOAP message ที่เซิร์ฟเวอร์ส่งกลับมามาใช้งานต่อไป

PHP เว็บเซอร์วิสโดยใช้ NuSOAP

NuSOAP เป็นกลุ่มของ PHP คลาสที่อนุญาตให้ผู้ใช้ส่งและรับ SOAP message บน HTTP โปรโตคอล NuSOAP นั้นเดิมชื่อ SOAPx4 ได้ถูกพัฒนาเป็นแกนกลางของหลายเว็บเซอร์วิส toolkit สำหรับ PHP ข้อดีของ NuSOAP คือ NuSOAP ไม่ใช่ส่วนเพิ่มเติม PHP แต่ได้ใช้คลาสพื้นฐานที่มีเมธอดที่มีประโยชน์ เช่น ตัวแปร ข้อมูล namespace และการจับคู่ตัวแปรที่เหมาะสมกับ namespace แบบต่าง ๆ การติดต่อกับเว็บเซอร์วิสทำได้โดยใช้ไคลเอนต์คลาส ชื่อ soapclient ที่อนุญาตให้ผู้ใช้กำหนดตัวเลือกต่าง ๆ เช่น HTTP authorization credential, ข้อมูล HTTP proxy และการจัดการกับการส่งและการรับ SOAP message ด้วยตัวเอง soapclient ใช้คลาสต่าง ๆ ช่วยในการส่งและการรับ SOAP message

การทำงานของ SOAP อาจทำงานโดยการส่งชื่อของงานที่ต้องการทำไปยังเมธอด call() ถ้าบริการมี WSDL ไฟล์ soapclient คลาสจะนำ URL ของ WSDL ไฟล์เป็นตัวแปรเริ่มต้นส่งไปยัง constructor และใช้ WSDL คลาสเพื่อทำการวิเคราะห์ WSDL ไฟล์และดึงข้อมูลออกมา WSDL คลาสมีเมธอดที่ใช้ดึงข้อมูลบน per-operation หรือ per-binding basis

เมื่อผู้ใช้ทำการเรียกใช้บริการ soapclient ใช้ข้อมูลเหล่านี้จาก WSDL ไฟล์เพื่อเข้ารหัสตัวแปรและสร้าง SOAP envelope เมื่อเมธอด call() ทำงาน soapclient คลาสจะใช้ soap_transport_http คลาสในการส่งข้อความและรับข้อความ ข้อความที่รับเข้ามาจะถูกวิเคราะห์โดยใช้ soap_parser คลาส



รูปภาพที่ 2.7 โค้ดแกรมอธิบายกระบวนการของการใช้ SOAP เว็บเซอร์วิสโดยใช้ NuSOAP

กระบวนการจะแตกต่างกันถ้าเว็บเซอร์วิสถูกใช้โดยไม่มีการทำ WSDL ไฟล์ URL ของบริการถูกส่งไปยัง constructor ของ soapclient คลาส การทำงานยังคงทำงานด้วยการใช้เมธอด call() ของอ็อบเจกต์ soapclient แต่รายละเอียดจะแตกต่างกัน เพราะในกรณีที่มี WSDL ไฟล์จะมีการส่งตัวแปรค่าเริ่มต้นไปยัง constructor ตัวแปรชนิดที่ส่งเป็นประจําสามารถถูกแทนได้ด้วยการใช้ soapval คลาสที่อนุญาตให้ผู้ใ้ปรับแต่ง serialization ของตัวแปรได้

2.6 ข้อดีของเว็บเซอร์วิส

เว็บเซอร์วิสเป็นเทคโนโลยีที่ประยุกต์มาจากเทคโนโลยีเดิมที่มีอยู่ เช่น แนวคิดการพัฒนาโปรแกรมเชิงวัตถุ การออกแบบระบบแบบกระจายศูนย์ ฯลฯ ดังที่ได้กล่าวมาแล้วข้างต้น

ดังนั้นโดยทั่วไปนั้นเว็บเซอร์วิสจะคล้ายกับแอปพลิเคชันบนเว็บ (web application) คือ โปรแกรมที่อยู่ในเว็บเซิร์ฟเวอร์ที่คอยให้บริการสิ่งที่ร้องขอ(request) แต่แทนที่จะส่งผลโดยส่งหน้าเพจ HTML เหมือนกับแอปพลิเคชันบนเว็บ กลับให้ค่าการคำนวณต่าง ๆ หรือข้อมูลที่ต้องการ กล่าวคือเว็บเซอร์วิสไม่ได้มีจุดประสงค์สำหรับบราวเซอร์และไม่มี User Interface (UI) แต่จะประกอบด้วยข้อมูลที่สามารถนำไปใช้ใหม่ได้ (reusable software components)

ปัญหาของเว็บแอปพลิเคชันในปัจจุบันคือการส่งข้อมูลไปกลับระหว่างเบราว์เซอร์ และเว็บเซิร์ฟเวอร์ และการแลกเปลี่ยนข้อมูลระหว่างเว็บเซิร์ฟเวอร์ด้วยตนเอง ซึ่งเว็บแอปพลิเคชันนั้นจะทำการส่งข้อมูลไปกลับเป็นรูปแบบของหน้าเว็บเพจหรือแท็ก HTML ที่ไม่สามารถสื่อสารถึงข้อมูลตัวแปรที่ต้องการได้เนื่องด้วยข้อจำกัดของรูปแบบแท็ก HTML ตามมาตรฐานองค์กร W3C ที่มีอยู่จำกัด การสร้างแท็กนั้นไม่สามารถทำได้ตามความต้องการของผู้ใช้ ซึ่งเว็บเซอร์วิสได้เสนอ XML มาใช้ในการแก้ปัญหาโดยให้ผู้ใ้สามารถสร้างแท็ก XML ขึ้นมาเองได้ตามความต้องการของผู้ใช้

อีกทั้ง HTML นั้นสามารถให้มุมมองเพียงด้านเดียวแก่ผู้เข้ามาเยี่ยมชม กล่าวคือไม่สามารถให้ผู้ใช้อย่างใดคนหนึ่งสามารถมองเห็นเพจเดียวกันได้อย่างแตกต่างกันได้ ซึ่งในงานหลายอย่างต้องการคุณสมบัติดังกล่าว โดยเฉพาะอย่างยิ่งการทำธุรกิจบนอินเทอร์เน็ตที่จำเป็นจะต้องมีการจัดแบ่งกลุ่มลูกค้า เช่น การจัดกลุ่มลูกค้าที่เข้ามาดูสินค้าบนเว็บไซต์เพื่อให้ลูกค้าแต่ละคนได้มุมมองเฉพาะในสินค้าส่วนที่ตนเองสนใจเท่านั้น เป็นต้น ซึ่งในที่นี้ได้ทำการสร้างบริการประกาศข่าวให้ผู้ได้รับเฉพาะประกาศที่เกี่ยวข้องกับผู้ใช้เท่านั้น เช่น สมาชิกที่อยู่ในชั้นปี 4D นั้นก็จะได้รับประกาศที่ประกาศถึงทุกคนและประกาศที่ประกาศถึงกลุ่มชั้นปี 4D เท่านั้น เป็นต้น โดยผู้ให้บริการประกาศข่าวนั้นจะทำการส่งข้อมูลมาให้เว็บเซิร์ฟเวอร์ในรูปแบบของ XML เพื่อทำการนำไปประมวลผลและตอบกลับไปยังผู้ใช้ตามเงื่อนไขที่เว็บเซิร์ฟเวอร์ต้องการได้